

INNOVIBE MOBILITY

Task Management System (TMS)

Complete System Documentation & Handover Manual

PREPARED & IMPLEMENTED BY

IT Interns

VERSION

1.0.27

DATE

June 2026

TECHNOLOGY STACK

Next.js • React • Supabase •
PostgreSQL

Table of Contents

1. Executive Summary	3
1.1 Overview	
1.2 Objectives	
2. User Roles & Capabilities	3
2.1 Admin	
2.2 Department Head	
2.3 Employee	
3. System Architecture	4
3.1 Frontend Layer	
3.2 Backend Layer	
3.3 Realtime Layer	
4. Module-Wise Documentation	5
4.1 Authentication Module	
4.2 Profile Management Module	
4.3 Workforce Management Module	
4.4 Task Management Module	
4.5 Attendance & Session Tracking Module	
4.6 Logout Report System	
4.7 Leave Management Module	
4.8 Announcements Module	
4.9 Notifications Module	
4.10 Reports Module	
4.11 Dashboard Modules	
5. Realtime Architecture & Counts Context	7
6. Database Overview & Schema Design	7
7. Security & Row-Level Access Control (RLS)	8
8. Operational Workflows	9
9. Developer Handover Guide	10
9.1 Folder Structure	
9.2 Major Files to Maintain	
9.3 Database Migration Notes	
10. Future Scope & Enhancements	11
11. Conclusion	11

INNOVIBE MOBILITY: TASK MANAGEMENT SYSTEM (TMS)

COMPLETE SYSTEM DOCUMENTATION & HANDOVER MANUAL

1. Executive Summary

1.1 Overview

The INNOVIBE Mobility Task Management System (TMS) is an enterprise-grade role-based workforce management, attendance tracking, and task collaboration platform. It serves as the primary operational hub for organizational compliance, resource coordination, leaves approvals, daily check-in reporting, and company-wide notifications.

1.2 Objectives

- **Centralization:** Consolidate workforce tracking, task delegation, leave management, and session monitoring in one secure platform.
 - **Accuracy:** Eliminate manual tracking gaps through automated, time-zone locked (IST) logs.
 - **Productivity:** Enhance output with multi-assignee task accountability, subtask verification, and daily work submission reporting.
 - **Onboarding compliance:** Ensure mandatory employee compliance through automated step-by-step profile registration.
-

2. User Roles & Capabilities

The TMS platform enforces three distinct user access tiers, restricting data visibility and functionality according to organizational roles.

2.1 Admin

- **Workforce Directory:** Perform complete organization-wide CRUD operations for employees and department heads.
- **Department Oversight:** Create and delete departments, modify department codes, and configure custom check-in cutoff policies.
- **System-Wide Tasks:** Create and assign collaborative tasks to any user, including employees, department heads, and other administrators.
- **Leave Supervision:** Review and manage leave records organization-wide.
- **Global Broadcasts:** Dispatch announcements with file attachments and audio/voice notes to specific target audiences.

2.2 Department Head

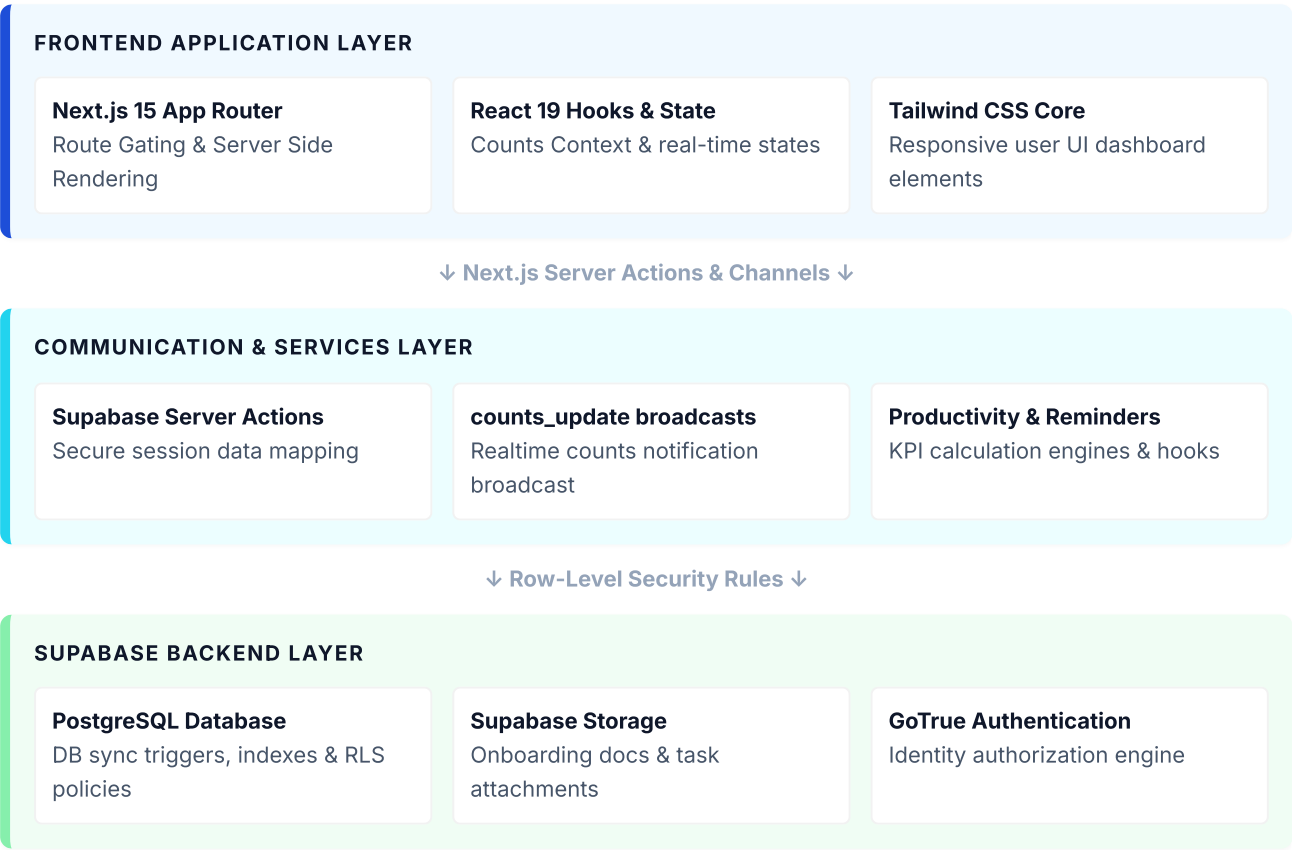
- **Department Management:** Full visibility and profile editing capabilities for employees within their specific department.
- **Localized Task Delegation:** Assign tasks to department members, track team deliverables, and manage subtask workflows.
- **Leave Approvals:** Inspect, approve, and reject department employees' leave applications with rejection reason entries.
- **Shift Supervision:** View login-out session histories and work-hours logs of department staff.

2.3 Employee

- **Work Execution:** Receive tasks, mark execution progress, comment on tasks, upload attachments, and create/complete task subtasks.
- **Leave Request Applications:** Submit digital leave applications, check approval status, and track leave balances.
- **Time Tracking compliance:** Perform shift check-ins, record daily work summaries, attach deliverables, and trigger check-outs.
- **Announcement Access:** View global and department-specific announcements, download files, and listen to voice broadcasts.

3. System Architecture

The platform is designed around a modern serverless architecture combining Next.js Server Components with a secure Supabase backend.



3.1 Frontend Layer

- **Framework:** Next.js 15 utilizing React 19 and the App Router directory structure.
- **Styling:** Tailwind CSS combined with Framer Motion for clean, functional micro-animations.
- **Client State:** React Context API (`TaskCountsContext`) handles real-time counter updates.

3.2 Backend Layer

- **Server Logic:** Next.js Server Actions execute backend queries directly, eliminating separate Web API layers.
- **File Storage:** Supabase Storage Buckets configured with Row-Level Security:

- `employee-profiles` & `department-profiles` for avatar images.
- `task-attachments` for task-specific file transfers.
- `onboarding-documents` for compliance paperwork.
- `announcements` for PDFs and voice messages.

3.3 Realtime Layer

- **Supabase Realtime:** Subscribes to PostgreSQL database write logs (Insert, Update, Delete).
- **Broadcast Channels:** Transmits counts updates (`counts_update`) to trigger async background updates on employee and manager dashboards without browser reloads.

4. Module-Wise Documentation

4.1 Authentication Module

- **Core Logic:** Implemented using Supabase Auth. Session routing is gated by a global Next.js middleware router that verifies tokens on incoming HTTP requests.
- **Login flow:** Supports email/password credentials. Resolves roles dynamically by querying database profiles in order of Admin, Department, and Employee.
- **Password Reset Workflow:** Gated reset requests are dispatched directly to the admin group as high-priority real-time notifications containing the target user's email ID.

4.2 Profile Management Module

- **Compliance Gate:** Gated via Next.js Middleware. Users with `onboarding_completed = false` are redirected to `/onboarding` to complete profile steps.
- **Completion Tracker:** Tracks profile fields completed (address, emergency contact, DOB, Aadhaar, PAN) and calculates a completion percentage shown on dashboards.
- **Uploaded Documents:** Stores official IDs in `onboarding-documents` with restricted admin-only select rights.

4.3 Workforce Management Module

- **Employee Creation:** Admins or Department Heads can create new accounts. An auth user is provisioned, and profile rows are created in `employees`.
- **Deletion Lifecycle:** Removing an employee performs a cascading cleanup of dependencies, ensuring no orphan data remains in `tasks`, `attendance`, `logout_reports`, or `leave_requests`.

4.4 Task Management Module

- **Multi-Assignee Support:** A single task can be linked to multiple collaborators through the `task_assignees` schema.
- **Backward Compatibility Triggers:** PostgreSQL database triggers (`sync_task_columns` and `sync_comment_columns`) ensure synchronization between modern cross-role schemas and legacy single-assignee layouts.
- **Collaboration Feed:** Enables file uploads (up to 20MB) and inline comments with real-time sync.
- **Status Recalculator:** Updates the task to `COMPLETED` only when all assignees complete their portions.

4.5 Attendance & Session Tracking Module

- **Late Check-in Logic:** Logs arrivals as `LATE` if check-in occurs after the department's configured `check_in_cutoff_time`.

- **IST-Aware Bounds:** Offsets database query times by +5:30 to prevent UTC timezone gaps from splitting morning sessions.
- **Double Check-in Flow:** Allows employees to check in and out multiple times per shift, generating consecutive active session logs.

4.6 Logout Report System

- **Session Termination:** Requires employees to submit daily work summaries, list completed/pending tasks, identify blockers, and attach deliverables.
- **Duration Calculation:** Calculates work session durations (e.g. `8h 15m`) and stores them inside `work_sessions` upon logout.

4.7 Leave Management Module

- **Request Submission:** Employees specify start date, end date, leave type, and reason.
- **Approval Chain:** Leave requests notify the department head and all admins. Approvals update balances; rejections require a reason string.

4.8 Announcements Module

- **Publishing Controls:** Gated for Admins and Department Heads. Targets can be set to the entire organization, specific roles, or specific departments.
- **Rich Media:** Supports attachments and voice recordings saved in `announcements` storage.

4.9 Notifications Module

- **Realtime Push:** Employs Supabase PostgreSQL replication to trigger real-time updates when notifications are inserted.
- **Routing Integration:** Clicking a notification redirects the user to the correct path (e.g., `/employee/tasks` or `/department/leave-approvals`).

4.10 Reports Module

- **Query Options:** Filters records by employee, department, and custom date ranges.
- **Export Options:** Generates PDF reports using jsPDF and Excel documents using sheetJS.

4.11 Dashboard Modules

- **Admin Dashboard:** Tracks overall active sessions, pending leaves, open tasks, and active staff.
- **Department Dashboard:** Tracks team attendance, late entries, pending approvals, and active tasks.
- **Employee Dashboard:** Displays shift check-in controls, task lists, and announcements.

5. Realtime Architecture & Counts Context

The TMS application provides a seamless, refresh-free user experience for task metric updates:

- **Trigger Event:** When a task is modified (created, accepted, completed, or subtask toggled), the backend Server Action calls `broadcastTaskCounts()`.
- **Channel Broadcast:** The function opens a Supabase broadcast channel named `public:tasks_counts` and sends a `counts_update` event.
- **Client Subscription:** The frontend `TaskCountsProvider` receives the event.

- **Data Sync:** The client calls `/api/tasks/counts` in the background to update dashboard counters without page reloads.

6. Database Overview & Schema Design

6.1 Table Definitions

- **users (Legacy / Core Profiles):** Links to `auth.users`, holds global designations and profile photo paths.
- **admins** : Stores system administrator profiles.
- **departments** : Defines organizational departments, check-in cutoff policies, and department codes.
- **employees** : Defines employee contracts, department mappings, and onboarding status.
- **tasks** : Contains core task fields (title, description, `created_by`, priority, `due_date`).
- **task_assignees** : Connects tasks to multiple assignees, tracking individual status.
- **task_subtasks** : Breaks tasks into subtasks with toggle states.
- **attendance** : Daily attendance records (`PRESENT` , `LATE` , `HALF_DAY` , `LEAVE` , `ABSENT`).
- **work_sessions** : Logs check-in/check-out login periods and links to logout reports.
- **logout_reports** : Daily work submissions, completed/pending tasks, and attachments.
- **leave_requests** : Holds leave requests, dates, and approval details.
- **announcements** : Organizational notifications with rich media and voice notes.
- **notifications** : Real-time user notification logs.

7. Security & Row-Level Access Control (RLS)

Database safety is enforced at the database level using PostgreSQL Row-Level Security (RLS) policies:

- **Admin Access:** Admins bypass RLS checks across directories, tasks, and configurations.
- **Department Scope:** Department Heads can read profiles, sessions, attendance, and leaves of employees belonging to their department ID.
- **Employee Scope:** Employees can only select, insert, or update rows matching `auth.uid() = employee_id` or `auth.uid() = user_id`.
- **Storage Rules:** Users must be authenticated to write files to buckets, and deleting attachments is restricted to the file owner.

8. Operational Workflows

8.1 Leave Application Workflow

- **Step 1:** Employee submits leave application via the dashboard.
- **Step 2:** System writes record to `leave_requests` and triggers database notifications.
- **Step 3:** Department Head and Admins receive alerts in real time.
- **Step 4:** Department Head approves or rejects the request (with notes).
- **Step 5:** Database record updates, and the employee is notified.

8.2 Task Assignment Workflow

- **Step 1:** Admin or Department Head fills out the task creator form, selecting multiple assignees.
- **Step 2:** System creates the task record and maps assignees inside `task_assignees`.
- **Step 3:** Collaborators receive instant notifications with links to their task board.
- **Step 4:** Collaborators accept, comment on, or block the task.
- **Step 5:** System recalculates overall status dynamically.

8.3 Attendance Tracking Workflow

- **Step 1:** Employee clicks "Check In" on dashboard.
- **Step 2:** System checks the current time against the department's cutoff time.
- **Step 3:** Record is created in `attendance` (marked `PRESENT` or `LATE`).
- **Step 4:** An active work session is initialized.
- **Step 5:** Management receives a real-time activity log notification.

8.4 Logout Report Workflow

- **Step 1:** Employee triggers check-out by completing the logout form.
- **Step 2:** System uploads any attached deliverables.
- **Step 3:** Active session in `work_sessions` is closed, and duration is calculated.
- **Step 4:** The `attendance` record is marked `LOGGED_OUT`.
- **Step 5:** Work summaries are stored in `logout_reports` and made visible to managers.

9. Developer Handover Guide

9.1 Folder Structure

```
TMS-main/
├── src/
│   ├── app/                # Next.js App Router Pages & API routes
│   │   ├── actions/        # Server Actions (auth, tasks, leaves, logouts)
│   │   ├── admin/          # Admin Dashboard routes
│   │   ├── department/     # Department Head Dashboard routes
│   │   └── employee/       # Employee Dashboard routes
│   ├── components/         # Reusable React UI Components
│   ├── context/            # Global Contexts (TaskCountsContext)
│   └── lib/                # Core utilities, Supabase Server/Client
├── migrations/             # Local SQL schema definitions
└── public/                 # Static assets and Service Worker configurations
```

9.2 Major Files to Maintain

- `src/app/actions/tasks.ts` : Handles multi-assignee task distribution, status recalculations, subtasks, and real-time broadcasts.
- `src/app/actions/logout.ts` : Controls session termination, duration logic, and daily deliverables uploads.
- `src/lib/supabase/client.ts` & `server.ts` : Client and server-side Supabase client initializers.

- `src/context/TaskCountsContext.tsx` : Real-time state manager for task metrics.

9.3 Database Migration Notes

When deploying to a new Supabase environment, run the SQL migrations in the following order:

- `supabase-schema.sql` (Base tables and Roles setup)
- `supabase-phase2-migration.sql` (Profile updates & Storage buckets)
- `supabase-phase3-migration.sql` (Attendance, Leaves, & Holidays tables)
- `supabase-phase4-tasks.sql` (Tasks, comments, and attachments schema)
- `supabase-phase8-announcements.sql` (Voice note columns and attachment JSONB)
- `supabase-phase9-onboarding.sql` (Onboarding compliance extensions)
- `supabase-phase11-cross-role-tasks.sql` (Cross-role columns and triggers)
- `supabase-phase12-sessions.sql` (Sessions & Logout reports schema)
- `supabase-task-collaboration.sql` (Multi-assignees tables and RLS)

10. Future Scope & Enhancements

- **Push Notifications:** Configure Web Push Notifications for system alerts.
- **Performance Analytics:** Generate monthly employee productivity scoring matrices.
- **Shift Scheduling:** Implement shift rotations and department head schedules.
- **Mobile Apps:** Package pages into Android/iOS bundles using Capacitor.

11. Conclusion

The INNOVIBE Mobility Task Management System (TMS) provides a reliable, secure environment for managing workforce workflows, task tracking, and attendance logs. By leveraging Next.js 15 Server Actions and Supabase PostgreSQL features, the system provides high performance, secure permission scopes, and real-time synchronization, ready for operational rollout.